

DA2404 - Artificial Intelligence Lab

End Sem - Coding Assignment

24 Apr 2026

Introduction

This is a coding assignment on concepts covered recently - Hidden Markov Models, Search & Reinforcement Learning. This assignment consists of multiple problem statements. The basic algorithms for each are provided as they have already been solved in class. Your task would be to make design choices or devise heuristics that improve performance of system. Each problem consists of public tests for you to understand the system. Scores will be based on hidden tests that evaluate your design performance.

Problem 1: Hidden Markov Models

This code implements a basic Intrusion Detection System, using a Hidden Markov Model. Network log data contains failed logins, data transfer, and DNS queries, from which suspicious patterns are to be identified. The folder contains the following files.

1. `dataset.py`: Generates synthetic data for failed logins and network traffic metrics
2. `hmm_core.py`: Contains full implementations of HMM algorithms - Forward, Viterbi and Baum-Welch
3. `pipeline.py`: Is the code to be completed. You should encode the observations, build HMM models and detect intrusions using log likelihood

4. run.py: Ties everything together, by generating data, encoding logs, building and training model

Evaluation will be done using hidden tests on log_likelihood, robustness, stability, training and inference time for multiple datasets.

Problem 2: Search

This code solves the 8 puzzle. Ref: https://en.wikipedia.org/wiki/15_puzzle. The task is to arrange the puzzle in a specific order, from a given starting state. Informed search algorithms with good heuristics perform optimally on such problems. Repository contains the following files.

1. puzzle.py: Contains rules and mechanics of the 8 puzzle. Has goal state, move logic and neighbour generation
2. astar.py: Implementation of A* algorithm
3. heuristics.py: Is the code to be implemented. Devise suitable heuristics that enable A* to reach the goal faster, with minimal node expansion and time

Evaluation will be done using hidden tests on cost, nodes expanded and time elapsed for search.

Problem 3: Reinforcement Learning

Contains code for training and evaluating an RL agent to play the game of Tic Tac Toe using Q learning. Contains the following files

1. tictactoe.py: Defines environment and game rules. Contains game logic, move validation and winner detection
2. agents.py: Contains logic for two agents, Random agent and Q learning agent
3. opponents.py: Contains random, defensive and aggressive opponents to evaluate agent performance

4. `design.py`: Is the code to be completed. Add feature extraction, reward function and epsilon scheduling choices
5. `training.py`: Contains training and evaluation functions for the agent

Evaluation will be done using hidden tests on win rate against various opponents.

Important Notes

The code template is shared in GitHub classroom. Submit before deadline.

1. Complete the assignment using plain Python with simple packages like NumPy and SciPy.
2. The usage of any ML packages (like sklearn, statsmodels, PyTorch , TensorFlow etc.) or built in ML libraries will make submission invalid.
3. Your code will be used to validate your predictions. Ensure that you do not alter the class structure from the one given in template.
4. Your learnings and observations from solving this assignment are important. Summarise them in `report.txt`